```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained('Qwen/Qwen2-0.5B', dty
model = AutoModelForCausalLM.from_pretrained('Qwen/Qwen2-0.5B',

messages = ["The Eiffel Tower is located in"
]

inputs = tokenizer(messages, return_tensors='pt')

outputs = model(**inputs)
print(outputs.logits.shape)

```

```
Loading weights: 100%  290/290 [00:07<00:00, 72.86it/s, Materializing param=model.norm.we
torch.Size([1, 8, 151936])
```

torch.Size([1, 26, 151936])-->(B, N_TOKENS, VOCAB_SIZE)

```
last_token = outputs.logits[0, -1,:]
token_id = torch.argmax(last_token)
decode = tokenizer.decode(token_id)
print(f"Token id: {token_id}")
print(f"Decoded to text: {decode}")
```

```
Token id: 12095
Decoded to text:  Paris
```

```
tokenizer.decode(torch.argmax(outputs.logits, dim=-1))
```

```
[' following-el Tower is the in Paris']
```

```
1 print(model)
```

```
Qwen2ForCausalLM(
  (model): Qwen2Model(
    (embed_tokens): Embedding(151936, 896)
    (layers): ModuleList(
      (0-23): 24 x Qwen2DecoderLayer(
        (self_attn): Qwen2Attention(
          (q_proj): Linear(in_features=896, out_features=896, bias=T
          (k_proj): Linear(in_features=896, out_features=128, bias=T
          (v_proj): Linear(in_features=896, out_features=128, bias=T
          (o_proj): Linear(in_features=896, out_features=896, bias=F
        )
        (mlp): Qwen2MLP(
          (gate_proj): Linear(in_features=896, out_features=4864, bi
          (up_proj): Linear(in_features=896, out_features=4864, bias
          (down_proj): Linear(in_features=4864, out_features=896, bi
          (act_fn): SiLUActivation()
        )
        (input_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
        (post_attention_layernorm): Qwen2RMSNorm((896,), eps=1e-06)
      )
    )
    (norm): Qwen2RMSNorm((896,), eps=1e-06)
    (rotary_emb): Qwen2RotaryEmbedding()
  )
  (lm_head): Linear(in_features=896, out_features=151936, bias=False
)
```

```
1 model.model.layers[12].mlp
```

```
Qwen2MLP(
  (gate_proj): Linear(in_features=896, out_features=4864,
bias=False)
  (up_proj): Linear(in_features=896, out_features=4864, bias=False)
  (down_proj): Linear(in_features=4864, out_features=896,
bias=False)
  (act_fn): SiLUActivation()
)
```

```
1 captured = {}
2 layers = [9, 10, 11, 12, 13, 14]
3
4 def get_hook(layer_id):
5   def hook(module, input, output):
6     captured[layer_id] = output.cpu().detach()
7   return hook
8
9 handles = [model.model.layers[n].mlp.register_forward_hook(get_
10
```

```
from datasets import load_dataset
ds = load_dataset("wikitext", "wikitext-103-raw-v1", split="tra
```

```
1
```

```
text = [text for text in ds['text'] if text.strip()]
len(text)
```

```
338
```

```
store= {9: [],
        10: [],
        11: [],
        12: [],
        13: [],
        14 :[]
           }

for i in text:
  tokenids = tokenizer(i,
                       return_tensors='pt',
                       truncation=True,
                       max_length=128)
  with torch.no_grad():
    y_pred = model(**tokenids)

  for layer in store:
    store[layer].append(captured[layer])

for handle in handles:
  handle.remove()
```

```
len(store[12])
```

```
338
```

```
store[12][0].shape
```

```
torch.Size([1, 8, 896])
```

```
activation_9 = torch.cat([t.squeeze(0) for t in store[9]], dim=
activation_10 = torch.cat([t.squeeze(0) for t in store[10]], di
activation_11 = torch.cat([t.squeeze(0) for t in store[11]], di
activation_12 = torch.cat([t.squeeze(0) for t in store[12]], di
activation_13 = torch.cat([t.squeeze(0) for t in store[13]], di
activation_14 = torch.cat([t.squeeze(0) for t in store[14]], di
```

```
activation_10.shape
```

```
torch.Size([18639, 896])
```

```
from torch import nn

class SAE(nn.Module):

  def __init__(self, in_dim, out_dim):
    super().__init__()
    self.encoder = nn.Linear(in_features=in_dim, out_features=o
    self.decoder = nn.Linear(in_features=out_dim, out_features=
    self.relu = nn.ReLU()

  def forward(self, x: torch.Tensor):

    hidden = self.relu(self.encoder(x))
    reconstruction = self.decoder(hidden)
    return reconstruction, hidden
```

```
sparsity = SAE(in_dim=896, out_dim=3584)
sparsity
```

```
SAE(
  (encoder): Linear(in_features=896, out_features=3584, bias=True)
  (decoder): Linear(in_features=3584, out_features=896, bias=True)
  (relu): ReLU()
)
```

```
1 sparsity(activation_9)
```

```
(tensor([[ 0.1062, -0.0413, -0.4470,  ..., -0.0113,  0.1753,
-0.1350],
         [ 0.2217, -0.0190, -0.2152,  ..., -0.2472,  0.0129,
-0.2193],
         [ 0.0167, -0.2077, -0.1304,  ..., -0.0165,  0.0657,
-0.1014],
         ...,
         [ 0.1162, -0.0501,  0.1964,  ...,  0.0037,  0.0811,
-0.1451],
         [ 0.1943, -0.0536, -0.0357,  ...,  0.1702, -0.0023,
0.0485],
         [ 0.0856,  0.0458, -0.1269,  ..., -0.0063,  0.0236,
-0.0760]],
        grad_fn=<AddmmBackward0>),
 tensor([[0.0000, 0.0000, 0.0000,  ..., 0.0000, 0.0000, 0.0000],
         [0.0507, 0.0283, 0.0453,  ..., 0.2137, 0.1432, 0.0270],
         [0.0000, 0.0459, 0.0212,  ..., 0.0049, 0.0283, 0.0055],
         ...,
         [0.1223, 0.0295, 0.0900,  ..., 0.0764, 0.0933, 0.0468],
         [0.1168, 0.1181, 0.0458,  ..., 0.1624, 0.0000, 0.1471],
         [0.0484, 0.0000, 0.0695,  ..., 0.0263, 0.0808, 0.0292]],
        grad_fn=<ReluBackward0>))
```

```
1 from torch.utils.data import DataLoader
```

```
1 train_dataloader = DataLoader(activation_9, batch_size=256)
2 tok = next(iter(train_dataloader))
3 tok.shape
```

```
torch.Size([256, 896])
```

```
1 tok.shape
```

```
torch.Size([256, 896])
```

```
1 loss_fn = nn.MSELoss()
2 optimizer = torch.optim.Adam(sparsity.parameters(), lr=1e-4)
3 lambda_val  = 5e-1
4 for epoch in range(40):
5   total_loss = 0
6   for x in train_dataloader:
7     optimizer.zero_grad()
8     reconstrunction, hidden = sparsity(x)
9     loss = loss_fn(reconstrunction, x) + lambda_val * hidden.ab
10    loss.backward()
11    optimizer.step()
12    total_loss+=loss.item()
13  print(f"Epoch {epoch+1} loss: {total_loss / len(train_dataloa
```

```
Epoch 1 loss: 0.0118
Epoch 2 loss: 0.0113
Epoch 3 loss: 0.0108
Epoch 4 loss: 0.0104
Epoch 5 loss: 0.0101
Epoch 6 loss: 0.0098
Epoch 7 loss: 0.0095
Epoch 8 loss: 0.0093
Epoch 9 loss: 0.0090
Epoch 10 loss: 0.0088
Epoch 11 loss: 0.0086
Epoch 12 loss: 0.0084
Epoch 13 loss: 0.0082
Epoch 14 loss: 0.0081
Epoch 15 loss: 0.0079
Epoch 16 loss: 0.0078
Epoch 17 loss: 0.0077
Epoch 18 loss: 0.0075
Epoch 19 loss: 0.0074
Epoch 20 loss: 0.0073
Epoch 21 loss: 0.0072
Epoch 22 loss: 0.0071
Epoch 23 loss: 0.0070
Epoch 24 loss: 0.0069
Epoch 25 loss: 0.0068
Epoch 26 loss: 0.0067
Epoch 27 loss: 0.0066
Epoch 28 loss: 0.0066
Epoch 29 loss: 0.0065
Epoch 30 loss: 0.0064
Epoch 31 loss: 0.0063
Epoch 32 loss: 0.0063
Epoch 33 loss: 0.0062
Epoch 34 loss: 0.0061
Epoch 35 loss: 0.0061
Epoch 36 loss: 0.0060
Epoch 37 loss: 0.0060
Epoch 38 loss: 0.0059
Epoch 39 loss: 0.0058
Epoch 40 loss: 0.0058
```

```
1
2 with torch.no_grad():
3   batch = next(iter(train_dataloader))
4   _, hidden = sparsity(batch)
5   sparsity_ratio = (hidden == 0).float().mean()
6   print(f"Sparsity: {sparsity_ratio:.4f}")
```

```
Sparsity: 0.8663
```

```
with torch.no_grad():
  _, hidden = sparsity(activation_9)
n_0 = hidden[:, 0]
top10 = torch.topk(n_0, 10)
print(top10.indices)
```

```
tensor([12729, 12730, 12765, 12761, 12472, 13656, 12724, 13654, 1276
```

```
lengths = torch.tensor([t.shape[1] for t in store[9]])
```

```
boundaries = torch.cumsum(lengths, dim=0)
print(boundaries[:10])
```

```
tensor([  8, 136, 246, 362, 368, 496, 624, 715, 721, 849])
```

```
top10
```

```
torch.return_types.topk(
values=tensor([0.6811, 0.6297, 0.6265, 0.5415, 0.4999, 0.4978,
0.4796, 0.4559, 0.4455,
        0.4344]),
indices=tensor([12729, 12730, 12765, 12761, 12472, 13656, 12724,
13654, 12766, 13653]))
```

```
index = []
for i in top10.indices:
  index.append((boundaries>i).nonzero()[0])
index
```

```
[tensor([233]),
 tensor([233]),
 tensor([234]),
 tensor([234]),
 tensor([230]),
 tensor([243]),
 tensor([233]),
 tensor([243]),
 tensor([234]),
 tensor([243])]
```

```
for n in index:
  print(text[n.item()])
```

```
h established players . The first deal General Manager Scott Howson m

h established players . The first deal General Manager Scott Howson m

v had failed to live up to expectations in Columbus , playing in only

v had failed to live up to expectations in Columbus , playing in only

 – 25 – 5 start , Head Coach Scott Arniel was fired and replaced by A

ing Rick Nash , though they were not actively shopping him . Howson s

h established players . The first deal General Manager Scott Howson m

ing Rick Nash , though they were not actively shopping him . Howson s

v had failed to live up to expectations in Columbus , playing in only

ing Rick Nash , though they were not actively shopping him . Howson s
```

```
1
```

```
[233, 233, 234, 234, 230, 243, 233, 243, 234, 243]
```

```
sd_idx = []
idx = []
for neuron in hidden:
  t3 = torch.topk(neuron, 3)
  idx.append([n.item() for n in t3.indices])

  sd = [(boundaries>i).nonzero()[0] for i in t3.indices]
  sd_idx.append(tuple(n.item() for n in sd))

result = dict(zip(sd_idx, idx))
result
```

```
{(6, 19, 41): [538, 1643, 3428],
 (2, 7, 3): [212, 678, 246],
 (23, 1, 3): [1971, 16, 251],
 (6, 1, 11): [574, 115, 1057],
 (11, 2, 7): [1034, 212, 678],
 (1, 40, 11): [43, 3338, 1100],
 (22, 40, 37): [1915, 3365, 3085],
 (22, 37, 35): [1915, 3177, 2953],
 (7, 2, 40): [678, 212, 3366],
 (7, 23, 35): [711, 2093, 2993],
 (7, 22, 37): [712, 1946, 3147],
 (23, 26, 38): [1988, 2253, 3201],
 (1, 16, 40): [73, 1438, 3327],
```

```
(6, 30, 23): [544, 2565, 2051],
(9, 2, 10): [723, 207, 915],
(31, 40, 41): [2683, 3417, 3473],
(10, 32, 41): [950, 2757, 3484],
(42, 10, 35): [3556, 954, 3047],
(2, 38, 42): [229, 3250, 3580],
(1, 7, 29): [118, 651, 2531],
(18, 24, 38): [1545, 2179, 3233],
(41, 9, 23): [3505, 783, 2014],
(6, 14, 28): [568, 1279, 2365],
(23, 29, 11): [1988, 2420, 990],
(38, 14, 32): [3247, 1313, 2812],
(2, 23, 35): [148, 2077, 3027],
(2, 30, 18): [229, 2636, 1545],
(4, 38, 23): [364, 3211, 2027],
(2, 37, 2): [177, 3120, 241],
(16, 37, 2): [1412, 3098, 206],
(40, 21, 2): [3377, 1781, 206],
(22, 9, 16): [1911, 721, 1483],
(35, 16, 24): [2953, 1392, 2158],
(22, 29, 35): [1936, 2439, 2939],
(22, 24, 10): [1846, 2103, 929],
(22, 23, 35): [1902, 2017, 2959],
(10, 9, 41): [926, 841, 3538],
(5, 30, 10): [431, 2647, 869],
(40, 21, 30): [3336, 1809, 2654],
(28, 38, 13): [2347, 3255, 1134],
(28, 1, 2): [2375, 130, 177],
(30, 16, 3): [2651, 1484, 333],
(1, 13, 28): [64, 1123, 2412],
(14, 37, 19): [1271, 3183, 1722],
(11, 9, 38): [1045, 759, 3208],
(29, 29, 27): [2501, 2509, 2284],
(5, 14, 21): [427, 1301, 1785],
(14, 30, 2): [1338, 2557, 168],
(29, 14, 41): [2421, 1263, 3521],
(29, 30, 14): [2500, 2623, 1259],
(41, 23, 21): [3505, 1970, 1778],
(5, 3, 19): [398, 265, 1718],
(16, 40, 21): [1402, 3341, 1778],
(10, 18, 38): [855, 1545, 3201],
(24, 31, 11): [2114, 2683, 1036],
(40, 28, 28): [3343, 2323, 2327],
(16, 14, 37): [1484, 1248, 3129],
(41, 24, 22): [3492, 2118, 1904],
(9, 2, 7): [748, 204, 644],
```

```
1 tokenizer.decode([678])
```

```
' all'
```

```
sentence_idx = (boundaries > 1988).nonzero()[0].item()
print(text[sentence_idx])
```

```
PlayStation Official Magazine – UK praised the story 's blurring of
```

```
from collections import Counter

all_sentences = [idx for key in result.keys() for idx in key]
cm = Counter(all_sentences).most_common(10)
```

```
r_s = ''
for i in cm:
  print(text[i[0]])

```

```
The game takes place during the Second Europan War . Gallian Army S

The anime 's title was inspired by the principle purpose of the Nam

The Tower Building of the Little Rock Arsenal , also known as U.S.

Partly due to these events , and partly due to the major losses in

Two manga adaptations were produced , following each of the game 's

For several years the arsenal , which was owned by the federal gove

This movement is prompted by the feeling that pervades the citizens

The item was intended simply as a piece of news , but telegraph lin

The game 's battle system , the BliTZ system , is carried over dire

Senjō no Valkyria 3 : Unrecorded Chronicles ( Japanese : 戦場のヴァル
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
r_s
```

```
' The game takes place during the Second Europan War . Gallian Army
Squad 422 , also known as " The Nameless " , are a penal military u
nit composed of criminals , foreign deserters , and military offend
ers whose real names are erased from the records and thereon offici
ally referred to by numbers . Ordered by the Gallian military to pe
rform the most dangerous missions that the Regular Army and Militia
will not do , they are nevertheless up to the task , exemplified by
their motto , Altaha Abilia , meaning " Always Ready . " The three
main characters are No.7 Kurt Irving , an army officer falsely accu
sed of treason who wishes to redeem himself ; Ace No.1 Imca , a fem
```